ENGINEERING ONBOARDING

FaceByYou — Developer Onboarding

Everything a new backend engineer needs to get productive: access, local setup, architecture, and how to ship code.

Repo: github.com/uvie2121/Facebyyou-api • AWS: Face by You (708761843741) / us-east-1
Stack: Python · FastAPI · AWS (EC2, S3, DynamoDB, Secrets Manager) · Terraform · GitHub Actions

1. Welcome & What You're Building

FaceByYou is an AI beauty & skincare companion. The backend (this repo) accepts a short face video/photo, runs AI analysis, and returns skin metrics, a skin-age estimate, and makeup recommendations, persisting each user's history. It's a FastAPI service running on EC2, backed by S3 (media) and DynamoDB (data), deployed automatically via GitHub Actions.

2. Access Checklist (request from the founder)

Before you can be fully productive, make sure you have:

Access	What / How
GitHub repo	Collaborator (or org member) on `uvie2121/Facebyyou-api` with write access.
AWS account	An **IAM user** in the Face by You account (708761843741) with appropriate permissions. **Do not** use root credentials.
AWS CLI keys	Create an access key under your IAM user → configure a named CLI profile (below).
Secrets	You generally never need raw secrets — they live in AWS Secrets Manager and load onto the server automatically.

3. Local Environment Setup

Prerequisites: Python 3.11+ (3.12 recommended), Git, and optionally Docker & Terraform.

```
# 1. Clone
git clone https://github.com/uvie2121/Facebyyou-api.git
cd Facebyyou-api

# 2. Virtual environment + dependencies
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt

# 3. Local config
cp .env.example .env          # safe local defaults; never commit .env
```

```
# 4. Run the API
uvicorn app.main:app --reload
# Open http://localhost:8000/docs for interactive API docs
```

Get a dev login token & call the API

```
TOKEN=$(curl -s -X POST localhost:8000/api/v1/auth/dev-login \
  -H 'content-type: application/json' \
  -d '{"email":"you@facebyyou.ai"}' | python3 -c 'import sys,json;print(json.load(sys.stdin)["access_tok

curl localhost:8000/api/v1/users/me -H "Authorization: Bearer $TOKEN"
```

Configure AWS CLI (for infra/debugging)

```
aws configure --profile facebyyou   # paste your IAM access key + secret, region us-east-1
export AWS_PROFILE=facebyyou
aws sts get-caller-identity          # should show account 708761843741
```

4. Repository Map

Path	What lives there
`app/main.py`	FastAPI app, middleware, request logging, error handling.
`app/core/`	Config (env vars), JSON logging, JWT auth/security.
`app/models/schemas.py`	Pydantic request/response models — the API contract.
`app/services/`	Business logic: S3 uploads, DynamoDB access, `face_analysis` (AI seam).
`app/api/routes/`	Endpoints: health, auth, users, uploads, analysis.
`infra/terraform/`	All AWS infrastructure as code (S3, DynamoDB, IAM, EC2, etc.).
`deploy/`	EC2 bootstrap, systemd unit, Nginx config.
`.github/workflows/`	CI (lint + test) and Deploy (to EC2 via SSM).
`tests/`	Offline tests (AWS calls are mocked).

5. API Endpoints

Method	Path (prefix `/api/v1`)	Purpose
GET	/health	Liveness probe
POST	/auth/dev-login	Dev-only JWT (disabled in prod)
GET / PUT	/users/me	Read / update profile
POST	/uploads/presign	Get a temporary S3 upload link
POST	/analysis	Run analysis on an uploaded object

GET	/analysis & /analysis/{id}	History & single result

6. How Deployment Works (CI/CD)

You don't manually SSH anywhere. The flow is automatic and key-less:

1. Open a PR → **CI** runs `ruff` (lint), `pytest` (tests), and a Docker build.
2. Merge to `main` → **Deploy** workflow runs.
3. GitHub Actions assumes an AWS role via **OIDC** (no stored keys) and uses **SSM Run Command** to tell the EC2 box to pull the latest code, reinstall dependencies, restart the service, and health-check itself.

> **Branching:** `dev` (active work) → `staging` (pre-prod) → `main` (production). Never push directly to `main`; use PRs.

Manual deploy / re-run

```
# From the GitHub UI: Actions → Deploy → "Run workflow" (choose a ref)
# Or trigger by merging to main.
```

7. Working With Infrastructure (Terraform)

> **Caution:** `terraform apply` changes live cloud resources and can cost money or delete data. Always run `terraform plan` first and have changes reviewed.

```
cd infra/terraform
export AWS_PROFILE=facebyyou
terraform init
terraform plan         # preview — safe, makes no changes
# terraform apply      # only when changes are approved
```

Key live resources today: EC2 `i-0e79c7070f1c72487`, bucket `facebyyou-uploads-dev`, tables `facebyyou-users-dev` / `facebyyou-analyses-dev`, secret `facebyyou-dev-app-config`, logs `/facebyyou/dev/api`.

8. Conventions & Quality Bar

- **Style:** run `ruff check . --fix` before committing; keep functions and files small and focused.
- **Tests:** add/extend tests in `tests/`; mock AWS so they run offline. `pytest` must pass.
- **Secrets:** never commit `.env`, keys, or credentials. Add new config to Secrets Manager and read it via `app/core/config.py`.
- **Logging:** use the structured logger; don't `print()`. Logs flow to CloudWatch.
- **The AI seam:** real models plug into `app/services/face_analysis.py` (`_run_inference`) — currently a deterministic placeholder.

9. Debugging the Live Server

```
# Open a shell on the instance WITHOUT SSH (uses SSM):
aws ssm start-session --target i-0e79c7070f1c72487 --profile facebyyou
```

```
# On the box:
sudo systemctl status facebyyou-api
sudo journalctl -u facebyyou-api -n 100 --no-pager
curl -s localhost:8000/api/v1/health
```

Logs are also in CloudWatch under `/facebyyou/dev/api`.

10. First-Week Checklist

- ☐ Get GitHub + AWS IAM access; configure the `facebyyou` CLI profile.
- ☐ Run the API locally and hit `/docs`.
- ☐ Read `app/main.py`, `app/api/routes/`, and `app/services/`.
- ☐ Make a tiny change on a branch, open a PR, watch CI pass.
- ☐ Run `terraform plan` (read-only) to understand the infrastructure.